TrustHome PH

System Architecture & Data Model

How the pieces fit: components, stack, routes, Firestore collections, the access boundary and query patterns of the current system.

Document	TH-02 · System Architecture & Data Model
Version / date	Rev. 2.0 · 4 October 2026
Factual baseline	TrustHome_Current_Progress_Report_2026-10-04.md (PR #42 `fixes` working copy, based on `features/v1-3-navigation-settings`; excludes `dev`)
Replaces	System Build Plan §0–2
Audience	Team Idle developers, Scrum Master, QA, panel/adviser reviewers, and AI coding assistants used by the team

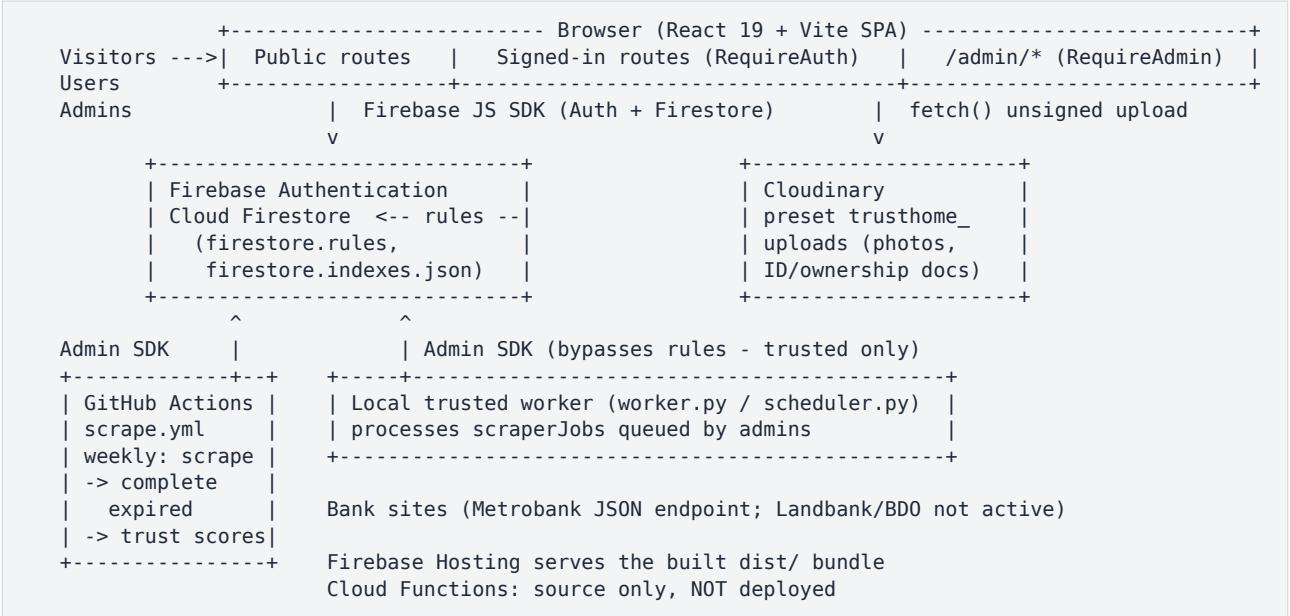
Evidence label	Meaning (never use these interchangeably)
[Implemented in source]	Code exists in the repository working copy. Not proof that it runs in production.
[Unit/emulator tested]	Covered by automated unit tests or the Firestore emulator suite.
[Reported working]	A team member observed it working in the browser against the test project.
[Deployed/live verified]	Deployed to trusthome-ph and confirmed working there.
[Known gap]	Does not yet meet the product claim. Do not present it as working.
[Roadmap] / **[Proposed]**	Planned future work / a recommendation in this document, not yet built.

Contents

1. Architecture Overview . . . 3
2. Technology Stack . . . 4
3. Source Layout . . . 5
4. Route Inventory . . . 5
5. Firestore Data Model . . . 6
5.1 Relationship diagram . . . 7
5.2 State values . . . 8
6. Access Boundary and Security Model . . . 8
7. Indexes and Query Patterns . . . 8

1. Architecture Overview

TrustHome is a **serverless single-page application**. The browser talks directly to Firebase Authentication, Cloud Firestore and Cloudinary. There is no always-on application server. Trusted work that must bypass security rules (scraping, scheduled maintenance, trust-score recomputation, backups) runs as **Python jobs using the Firebase Admin SDK**, triggered by GitHub Actions or a trusted local worker.



NOTE · Why this shape

- Firestore + Auth on the free Spark plan need no card; Cloud Functions and Firebase Storage need the Blaze billing plan, which the team has not enabled. That single constraint explains the Cloudinary choice, the client-side booking transaction and the Python/GitHub Actions jobs.
- Keeping the scraper outside the web app means a bank-site change can break the catalog refresh without affecting listings, bookings or disputes.

2. Technology Stack

Layer	Choice	Status	Notes / best practice
UI framework	React 19 SPA, React Router	**[Implemented in source]** **[Unit/emulator tested]**	Lazy-load route components; wrap the tree in an error boundary; keep business rules in `src/lib/` as pure, unit-testable functions
Build tool	Vite (`npm run dev/build/preview`)	**[Unit/emulator tested]**	Vendor chunk splitting removed the large-bundle warning; output in `dist/`
Auth	Firebase Authentication: email/password + Google	**[Implemented in source]**	Never build custom password storage; enforce authorization in rules, not only in route guards
Database	Cloud Firestore (`trusthome-ph`)	**[Implemented in source]** **[Deployed/live verified]** rules	Model for the queries you run; denormalize display fields; keep private fields in separate documents
File storage	Cloudinary, unsigned preset `trusthome_uploads`	**[Implemented in source]** **[Known gap]** delivery	Photos may be public; identity documents must move to authenticated delivery (TH-04 §6)
Hosting	Firebase Hosting	Last deploy 2026-10-01	SPA rewrite to `/index.html`; deploy only from a clean, tested build
Serverless functions	Firebase Functions (`functions/`)	**[Implemented in source]** not deployed	Requires Blaze + Cloud Build; do not claim callable/scheduled behavior is live
Trusted jobs	Python 3.10+, `requests`, `BeautifulSoup`, `firebase-admin`	**[Unit/emulator tested]** 28 tests	Run with least-privilege service account from a secret store
Scheduling	GitHub Actions (`scrape.yml`, weekly); local `scheduler.py` (24 h)	**[Implemented in source]**	Validate secret access and a controlled run before relying on it
Quality	Node test runner, ESLint, Firestore emulator, Python unittest	**[Unit/emulator tested]**	All four must pass before merge to the release branch

3. Source Layout

```
team-idle/
|-- src/
|   |-- App.jsx                route table (public / RequireAuth / RequireAdmin)
|   |-- firebase.js            single Firebase app init (project: trusthome-ph)
|   |-- uploadImage.js         Cloudinary upload helper (raw for PDF, auto/image otherwise)
|   |-- context/AuthContext.jsx listens to users/{uid}; exposes profile + role
|   |-- routes/                RequireAuth.jsx, RequireAdmin.jsx (fail-closed)
|   |-- pages/                 user-facing pages
|   |-- admin/                 admin workspaces (dashboard, listings, disputes, users, ...)
|   |-- components/            shared UI (cards, badges, nav, map, forms)
|   `-- lib/                   pure logic: booking.js, dispute.js, trustScore.js,
|                              listingValidation.js, notifications.js
|-- functions/index.js         callable confirmBooking (per-listing lock) + scheduled jobs - NOT deployed
|-- bank_scraper/              scraper.py, database.py, worker.py, scheduler.py, trust_scores.py,
|                              complete_expired_bookings.py, backup_firestore.py,
|                              restore_firestore.py, migrate_verification_documents.py,
|                              seed_demo_data.py, test_*.py, requirements.txt
|-- scripts/firestore-rules-smoke.mjs   emulator rule tests
|-- .github/workflows/scrape.yml        weekly: scrape -> complete expired -> trust scores
|-- firestore.rules  firestore.indexes.json  firebase.json
`-- docs/                                   these PDFs + progress reports
```

BEST PRACTICE · **Code organisation rules**

- Pages orchestrate; `src/lib/` decides. Overlap checks, status transitions, validation and scoring belong in `lib/` so they can be unit tested without a browser.
- One Firestore access pattern per concern (e.g. all booking writes go through `lib/booking.js`). Never write a status from a component directly.
- Status strings, collection names and route paths are constants in one module — never retype `"Confirmed"` in five files.
- When logic exists in two languages (trust score in JS and Python), add a shared fixture and a parity test.

4. Route Inventory

Routes are declared in `src/App.jsx`. User routes require authentication (`RequireAuth`); `/admin/*` routes use `RequireAdmin`, which requires a readable profile with `role: admin` and `status: active` and **fails closed** if the profile cannot be read. Route guards are a UX convenience — Firestore rules are the real security boundary.

Route	Access	Page / workflow
`/`	Public	Home and featured verified listings
`/browse`	Public	Listing search, filtering (purpose, city, price, type, details) and sorting
`/listings/:listingId`	Public (verified only)	Listing detail, trust signals, approximate location
`/bank-catalog`, `/bank-catalog/:propertyId`	Public	Bank property search; detail with loan estimate
`/profiles/:userId`	Public	Public profile and accountability summary
`/legal/:documentId`	Public	Legal, privacy and terms content
`/login`	Public	Sign in / register (email or Google)
`/listings/new`, `/listings/:listingId/edit`	Auth	Owner listing submission and edit/resubmission

Route	Access	Page / workflow
`/my-listings`, `/my-activity`, `/dashboard`	Auth	Owner listings and statuses; activity hub (listings, bookings, booking requests); owner metrics
`/listings/:listingId/book`	Auth	Renter booking request form
`/my-bookings`	Auth	Renter bookings: status, deposit reference, completion, review, dispute
`/booking-requests`, `/manage-bookings`	Auth	Owner request/availability views (also embedded in My Activity)
`/bookings/:bookingId/review`, `/bookings/:bookingId/dispute`	Auth	Rating form; dispute submission
`/notifications`, `/settings`, `/account`	Auth	Notification inbox; appearance/accessibility; account
`/saved-searches`	Redirect	Redirects to `/browse`; legacy records not deleted
`/admin`, `/admin/listings`, `/admin/disputes`, `/admin/users`	Admin	Dashboard; listing review queue; dispute resolution; user management
`/admin/notifications`, `/admin/bank-catalog`, `/admin/settings`	Admin	Admin inbox; queue scraper jobs; admin appearance settings

5. Firestore Data Model

Firestore is a document database: collections of JSON-like documents, no joins. Relationships are stored as IDs (`listingId`, `ownerId`, `renterId`). Public and private data are **split into separate documents** because Firestore rules secure whole documents, not individual fields.

Collection	Purpose and key fields	Intended access
`users/{uid}`	Private profile: name, email, `role` (user/admin), `status` (active/suspended), `createdAt`. Doc ID = Auth UID.	Owner and active admins; users cannot change own role/status
`publicProfiles/{uid}`	Public display fields only	Public read; owner/admin write
`listings/{listingId}`	`ownerId`, title, property type, purpose (rent/sale), rental term, city, price, price period, rooms, floor/lot area, amenities, availability, showing windows, `photoUrls`, coarse map location (5-char geohash, approximate flag), `verificationStatus`, review note, timestamps. **No exact address, no document URLs.**	Public reads verified only; owner reads own; admins review
`listingPrivate/{listingId}`	`ownerId`, exact address, `ownershipDocumentUrl`, `governmentIdUrl` (future: `documents` metadata for authenticated assets)	Active owner and admins only; tightly constrained create/update
`bookings/{bookingId}`	`listingId`, `renterId`, `ownerId`, start/end dates, `status`, display metadata, optional `depositReference`, optional address (not populated by current client path), dispute link/prior status, timestamps	Renter, owner, admins; transition rules restrict writes
`ratings/{ratingId}`	`bookingId`, reviewer, owner/listing, `score` 1–5, comment, `createdAt`	Public read; eligible renter creates after completion; admins moderate

Collection	Purpose and key fields	Intended access
disputes/{disputeId}	bookingId, reporter, owner, reason, status (Open/Founded/Dismissed), prior booking status, resolution notes, resolver, resolvedAt	Reporter and admins; constrained transitions
publicAccountability/{uid}	Count of founded disputes	Public read; admin write
trustScores/{listingId}	Cached score, booking/rating/fairness metrics, fairness label, manual-review flag, computedAt	Public read; trusted job/admin write only
notifications/{id}	recipientId (UID or __admins__), type, title, message, link, entity, read, createdAt	Recipient-scoped; admin broadcast types limited by rules
bankProperties/{bank_reference}	bank, reference no., title, location, price, floor/lot area, source URL, image, lowercase keyword prefixes, status active/delisted, firstSeen, lastSeen	Public reads active records; writes by trusted scraper/admin
scraperJobs/{jobId}	Requested banks, job status, result fields, timestamps	Admins create/read; trusted worker processes
savedSearches/{id}	Legacy records from the retired feature	Owner-scoped; no active UI

CAUTION · Field names

This table lists the fields the progress report documents. Before writing new code, confirm the exact field spelling in `firestore.rules` and `src/lib/`. Rules reject unexpected keys, so a misspelled field fails silently in the UI as a permission error.

5.1 Relationship diagram



5.2 State values

Entity	States	Allowed transitions (enforced in rules + lib)
Listing verification	pending, verified, changes requested, rejected	create → pending; admin: pending → verified / changes requested / rejected; owner edit of changes-requested → pending
Booking	Pending, Confirmed, Completed, Disputed	renter creates Pending; owner Pending → Confirmed; owner declines Pending (request removed); renter or expiry job Confirmed → Completed; renter dispute → Disputed; Dismissed restores prior status
Dispute	Open, Founded, Dismissed	reporter creates Open; admin Open → Founded / Dismissed
User	active, suspended	admin only
Bank property	active, delisted	trusted scraper only; never deleted
Scraper job	job status values set by worker.py (confirm exact strings in source)	admin creates; trusted worker updates

6. Access Boundary and Security Model

1. **Authentication** proves identity (UID). It does not grant any permission by itself.
2. **Profile** (`users/{uid}`) supplies `role` and `status`. Rules read it with `get()` to decide admin rights; suspended users lose write access.
3. **Ownership** fields (`ownerId`, `renterId`) are set at create time and are immutable afterwards.
4. **Document split**: anything a stranger must never see goes into a separate, owner/admin-only document (`listingPrivate`).
5. **State machines** are enforced in rules by comparing `resource.data.status` (before) with `request.resource.data.status` (after) and restricting `diff().affectedKeys()`.
6. **Trusted jobs** write derived data (trust scores, bank properties, auto-completion) with the Admin SDK; clients are denied.
7. **Deny by default**: any unmatched path is denied.

KNOWN GAP · Where the boundary is weaker than it looks

- Cloudinary URLs for ID and ownership documents are fetchable by anyone who has the URL. Firestore privacy of `listingPrivate` does not make the file private.
- The booking overlap check is client-side before the transaction; rules cannot see other bookings, so two simultaneous approvals can both succeed.

7. Indexes and Query Patterns

Composite indexes live in `firestore.indexes.json` and are deployed with `firebase deploy --only firestore:indexes`. A query that needs a missing index fails with a console link to create it — add it to the JSON file rather than clicking the link, so the index is version-controlled.

Query	Typical filters / order	Notes
Public browse	`verificationStatus == verified` + purpose/city/type, order by score or price	Trust order reads cached `trustScores`
Admin review queue	`verificationStatus == pending`, order by `createdAt`	Admin-only by rules

Query	Typical filters / order	Notes
Owner booking requests	`ownerId == uid` + `status`	Owner-scoped query must match rule conditions exactly or it is rejected
Overlap check before approval	`ownerId == uid`, `listingId`, `status == Confirmed`	Runs in the client before the transaction
Renter bookings	`renterId == uid`	
Notifications	`recipientId == uid` (or `__admins__` for admins), order by `createdAt desc`	
Bank catalog	`status == active` + bank / city / keyword prefix / price, paginated	Keyword search uses stored lowercase prefixes (`array-contains`)

BEST PRACTICE · Firestore modelling best practices applied

- Design documents around the screens that read them; duplicate small display fields (title, city, photo) into bookings so lists need one query.
- Queries must be provably allowed: a rule like `resource.data.ownerId == request.auth.uid` requires the query to filter on `ownerId == uid`.
- Use deterministic document IDs where a natural key exists (`metrobank_<ref>`, `trustScores/{listingId}`) for idempotent upserts.
- Use `serverTimestamp()` for every created/updated time; never trust client clocks for ordering or expiry.
- Paginate with `limit()` + `startAfter()`; never download a whole collection in the browser.
- Batch or transact multi-document changes (listing + listingPrivate; dispute + booking) so they succeed or fail together.
- Keep documents small (well under the 1 MiB limit); store files in Cloudinary and keep only references.